

Programming Assignment #01 Finding Roots of Equations

Finding the root of an equation $f(x) = 0$ is one of the fundamental problems in numerical analysis. Many real-world problems reduce to solving such equations, but closed-form analytical solutions are not always possible. In this module, we will learn how to use **iterative methods** for root-finding, and also learn the essential procedures for numerical method.

Generally, the program assignment will consist of 5 parts

- Problem definition
- Algorithm Design
- Implementation
- Test and Verification
- Analysis and Documentation

We will dive into details in the following.

1. The problem

We have learned the bisection, false position, and newton's methods in class. In this assignment, we will try to use the three methods to solve the equations with a simple root. All the equations will have one and only have one single root in the given interval.

Your application will take a set of command-line arguments in the following format. You can use your preferred function name to replace mySolver. The solver will have multiple arguments to assign the problem type, solver, and other required inputs.

```
>>> ./mySolver [problem type] [solver] [arguments]
```

Problem type

1. **POLY**: $c_n x^n + c_{n-1} x^{n-1} + \dots + c_2 x^2 + c_1 x + c_0 = 0$;
Where $p(x) = c_n x^n + c_{n-1} x^{n-1} + \dots + c_2 x^2 + c_1 x + c_0$
2. **COS**: $\cos x = c$; where $c \in [-1, 1]$, and $x \in [0, \pi]$
3. **SIN**: $\sin x = c$; where $c \in [-1, 1]$, and $x \in [-\pi/2, \pi/2]$
4. **TAN**: $\tan x = c$; where $c \in (-\infty, +\infty)$, and $x \in (-\pi/2, \pi/2)$
5. **EXP**: $e^x = c$; where $c \in (0, +\infty)$, and $x \in (-\infty, +\infty)$
6. **LOG**: $\ln x = c$; where $c \in (-\infty, +\infty)$, and $x \in (0, +\infty)$

Solver Type

1. **BiSec**: Bi-section
2. **FalsePos**: False Position
3. **Newton1**: Newton with analytic derivative
4. **Newton2**: Newton with numerical derivative
5. **Bracket-Newton1**: Newton-Bisection with analytical derivative
6. **Bracket-Newton2**: Newton-Bisection with numerical derivative

Arguments

1. **POLY**: $c_n \ c_{n-1} \ \dots \ c_1 \ c_0 \ a \ b$; $[a, b]$ is the given interval containing a single root.
2. **COS/SIN/TAN/EXP/LOG**: $c \ a \ b$

Note 1: We will not put a lot of efforts on double check the existence of the single root. You can just check if $f(a) \cdot f(b) \leq 0$. For $[a, b]$, you can choose reasonable setups, which can be a general $x \in [0, \pi]$

or a narrow one.

Note 2: If you want to control the way and step size to calculate the numerical derivative, just change the input arguments accordingly.

2. Algorithm Design

You can follow the class notes to design your C/C++ codes.

Be careful

- Always include stopping conditions to avoid an infinite loop including maximum iteration limit
- Implement a method that combines the stability of the bisection method with the fast convergence of Newton's method (Bracket-Newton). This will require a combined approach:
 - Use Newton's method when possible
 - Fallback to a bisection step whenever a Newton step leaves the interval or does not reduce the error
- Use Horner's algorithm to calculate $p(x)$ and $p'(x)$ as necessary
- Evaluate the root to the best possible precision (using **64**-bit floating point values, double)
- Carefully choose the value of Δx to calculate the numerical derivative. You can also set the step as an optional input argument.

3. Implementation

Work with your team to develop the code. You can use ChatGPT or other tools to help you, but the final code must be your team's own work.

4. Test and Verification

A python-based test script is preferred to automatically generate the test sets and verify the results. We need to generate both general (common) cases and corner cases. For general cases, you may generate random coefficients or constants as the inputs, but ensure the chosen interval brackets exactly one root.

You can start from the following code (might be "mySolver.exe" for windows)

```
import subprocess

def run_solver(problem, solver, args):
    cmd = ["/mySolver", problem, solver] + list(map(str, args))
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        print("Error:", result.stderr)
    return result.stdout.strip()

# Example: polynomial x^2 - x - 2 = 0, interval [1,3]
out = run_solver("POLY", "BiSec", [1, -1, -2, 1, 3])
print("Solver output:", out)
```

5. Analysis and Documentation

Carefully document what your team has done, and compare the performances of these solvers in terms of

- Stability and accuracy
- Convergence speed (time/iterations)
- Implementation/Computational Cost

- Etc.

6. Submission

Submit the following docs through the Canvas. Each member needs to submit individually, even the contents are the same.

- Report – PDF
- C/C++ code – .c/.cpp and a short description if you upload multiple files.
- Python script – .py

ECE 3340 – Programming Assignment 1

Finding Roots of Equations

Rubric

Team _____

Team work and design _____ / 10

Implementation _____ / 40

Bracket method _____ / 10

Newton _____ / 10

Newton-Bracket _____ / 10

Numerical Derivative _____ / 5

Horner's algorithm _____ / 5

Test and Verification _____ / 30

Scripting interface _____ / 10

General cases _____ / 15

Corner cases _____ / 5

Analysis and Documentation _____ / 20

Analysis _____ / 10

Documentation _____ / 10

Total _____ / 100